Object Oriented Programming Lab (CSEG1044) Experiment-09&10: Inheritance + Abstraction (Student Handout — Before Submission)

Tofik Ali

March 3, 2026

Contents

1 About this handout **1**

2 Experiment 09 (Syllabus) **1**

3 Experiment 10 (Syllabus) **1**

4 What you must do (minimum requirements) **2**
4.1 Task A — Inheritance + overriding (Exp-09) 2
4.2 Task B — Abstract class + interface (Exp-10) 2

5 Hints (read only if stuck) **2**

6 What to submit (MANDATORY) **3**

1 About this handout

This handout describes what to implement for inheritance, overriding, abstract classes, and interfaces. It intentionally avoids giving full code.

2 Experiment 09 (Syllabus)

- **Objective:** To implement inheritance and runtime polymorphism.
- **Problem Statement:** Create base and derived classes demonstrating inheritance.
- **Expected Outcome:** Students will understand code reusability.

3 Experiment 10 (Syllabus)

- **Objective:** To implement abstraction using abstract classes and interfaces.
- **Problem Statement:** Design programs using abstract classes and interfaces.

- **Expected Outcome:** Students will understand abstraction and multiple inheritance.

4 What you must do (minimum requirements)

4.1 Task A — Inheritance + overriding (Exp-09)

Create `InheritanceDemo.java`:

1. Create base class `Employee` with fields (`id`, `name`) and method `double calculatePay()`.
2. Create derived classes:
   - `SalariedEmployee extends Employee` (monthly salary)
   - `ContractEmployee extends Employee` (hourlyRate × hours)
3. Override `calculatePay()` in derived classes and use `@Override`.
4. Demonstrate runtime polymorphism:
   - store mixed objects in an `Employee[]` array,
   - call `calculatePay()` in a loop and print results.

4.2 Task B — Abstract class + interface (Exp-10)

Create `AbstractInterfaceDemo.java`:

1. **Abstract class part**
   - Create abstract class `EmployeeBase` with an abstract method `double calculateSalary()`.
   - Create at least two subclasses (example: `FullTimeEmployee`, `InternEmployee`) and implement salary logic.
2. **Interface part**
   - Create interface `Payable` with method `double amountDue()`.
   - Implement it in two classes (example: `Invoice` and `SalarySlip`).
   - Store objects in a `Payable[]` array and compute total due.

5 Hints (read only if stuck)

- Overriding happens when method name + parameters match and the subclass provides a new implementation.
- Use `@Override` to avoid accidental signature mismatches.
- Prefer interfaces to model capabilities; use abstract classes when you need shared state + partial implementation.

6 What to submit (MANDATORY)

Submit a single zipped folder containing:

- `InheritanceDemo.java`
- `AbstractInterfaceDemo.java`
- sample output for both programs
- observations (5–10 lines): dynamic dispatch; when to prefer interface vs abstract class

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed March 3, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed March 3, 2026).